

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая/Лабораторная работа

Выполнил:

Цветкова Татьяна

К3341

Проверил:

Добряков Д. И.

Санкт-Петербург 2026 г.

Задача

Тема: Домашнее задание №3. Тестирование API средствами Postman.

Вариант: «Платформа для фитнес-тренировок и здоровья».

Цель работы: реализовать тестирование разработанного в LP1 REST API средствами Postman, написать тесты внутри Postman и продемонстрировать один комплексный сценарий по основному процессу приложения.

Основной пользовательский сценарий: регистрация → вход → личный кабинет → обновление профиля → поиск тренировок с фильтрацией → просмотр случайной тренировки → создание плана тренировок → добавление тренировки в план → отметка выполнения → запись прогресса → получение сводной статистики → блог.

Ход работы

Тестирование выполнено в Postman поверх REST API, разработанного в рамках LP1. API запускается локально по адресу `http://localhost:3000/api`, базовый префикс задан переменной окружения `baseUrl`. Используется JWT-аутентификация: access-токен после регистрации/входа автоматически сохраняется в переменную коллекции `accessToken` и подставляется в Authorization-заголовок защищённых запросов.

1. Подготовка окружения

Созданы два артефакта: коллекция `FitnessPlatform.postman_collection.json` (схема Postman Collection v2.1.0) и окружение `FitnessPlatform.postman_environment.json`. В окружении заведены переменные `baseUrl`, `accessToken`, `refreshToken`, `userEmail`, `userPassword`, `userId`, `workoutId`, `planId`, `planItemId`. Большинство переменных заполняется автоматически в Tests-скриптах после соответствующих запросов и переиспользуется на следующих шагах.

2. Комплексный сценарий

Коллекция содержит 18 шагов, покрывающих основной процесс приложения и негативные сценарии:

1. POST /auth/register — регистрация нового пользователя; в pre-request скрипте генерируется уникальный email через timestamp, что позволяет запускать сценарий многократно без 409 Conflict.
2. POST /auth/login — вход созданным пользователем; токены сохраняются в переменные.
3. POST /auth/login с неверным паролем — негативный кейс, ожидается 401 Unauthorized.
4. GET /users/me — личный кабинет: профиль пользователя по сохранённому токену.
5. PATCH /users/me — обновление антропометрии и уровня подготовки.
6. GET /workouts — общий список тренировок, проверка структуры и обязательных полей.
7. GET /workouts?type=cardio&level=beginner&maxDuration=30 — поиск с фильтрацией; тесты проверяют, что все элементы результата соответствуют фильтру; id случайной тренировки сохраняется в переменную workoutId.
8. GET /workouts/{workoutId} — страница выбранной тренировки с видео и инструкциями.
9. GET /workouts/{несуществующий UUID} — негативный кейс, ожидается 404 Not Found.
10. POST /workout-plans — создание личного плана тренировок; planId сохраняется.
11. POST /workout-plans/{planId}/items — добавление в план ранее выбранной тренировки; planItemId сохраняется.
12. PATCH /workout-plans/{planId}/items/{planItemId}/complete — отметка элемента выполненным.
13. POST /progress — запись фактически выполненной тренировки с длительностью, калориями и рейтингом.
14. POST /progress с рейтингом 99 — негативный кейс на валидацию, ожидается 422 Validation Error с подробностями по полю rating.
15. GET /progress/stats — агрегированная статистика прогресса пользователя.

- 16. GET /blog — список опубликованных постов блога о здоровье и питании.
- 17. GET /users/me без Authorization-заголовка — негативный кейс, ожидается 401.
- 18. POST /workouts от обычного user — проверка ролевой модели, ожидается 403 Forbidden.

3. Что проверяется в Tests-скриптах

- корректные HTTP-статусы (200, 201, 401, 403, 404, 422);
- наличие и типы обязательных полей в ответах (id, токены, поля профиля и тренировки);
- согласованность данных между шагами (один и тот же пользователь, тренировка, план);
- корректность фильтрации (все элементы результата удовлетворяют параметрам поиска);
- отсутствие утечки passwordHash в ответах;
- время ответа (для логина < 1000 мс);
- детали ошибок валидации (массив details с полем и constraints);
- ролевая модель (user не имеет права создавать тренировки).

4. Запуск

В Postman импортируются файлы коллекции и окружения, выбирается окружение «Fitness Platform — local». Для пакетного запуска используется встроенный Collection Runner (кнопка Run): выполняются все 18 запросов последовательно, в итоговом отчёте отображается количество пройденных и упавших тестов. Альтернативно сценарий можно запустить из командной строки с помощью Newman:

```
newman run FitnessPlatform.postman_collection.json -e FitnessPlatform.postman_environment.json.
```

Вывод

В ходе домашнего задания подготовлен комплексный сценарий тестирования REST API средствами Postman. Сценарий покрывает основной пользовательский путь приложения — от регистрации до

получения статистики прогресса — и включает 18 шагов с цепочкой переменных, которые автоматически прокидываются между запросами через `pm.collectionVariables`.

Каждый запрос сопровождается Tests-скриптом, в котором с помощью `pm.test` и `chai`-подобных матчеров проверяются HTTP-статус, структура и содержимое ответа, соблюдение бизнес-правил (фильтрация, ролевая модель), а также обработка негативных кейсов (401, 403, 404, 422). Помимо «счастливого пути», в сценарий заложены проверки ошибок аутентификации, авторизации, валидации и поиска по несуществующему `id` — такие проверки гарантируют, что API корректно реагирует не только на ожидаемые, но и на ошибочные запросы.

Цель работы достигнута: реализовано тестирование API средствами Postman, написаны тесты внутри Postman, продемонстрирован комплексный сценарий основного бизнес-процесса приложения. Коллекция повторно запускаема (Run Collection / Newman), переиспользует переменные окружения и не требует ручной правки данных перед каждым прогоном.